

Distribution Strategy Module Implementation Requirements

v1.1 + v1.2 — Complete Specification

What this document is: Complete implementation instructions for the Distribution Strategy Module — a post-Phase-3 addition to the Through the Noise Monte Carlo Portfolio Simulation. This document combines v1.1 (four withdrawal strategies + Income Variability Report) and v1.2 (Minimum Annual Withdrawal floor + Strategy Info Bubbles) into a single paste-ready specification.

How to use it: Attach this PDF to your Claude Code session. Claude Code should read the full document before writing any code. All implementation instructions, code snippets, HTML, CSS, and JavaScript are ready to use directly.

Phase context: Phase 3 (input panel) is fully complete. Phase 2 (simulation engine) is fully complete. This module is a carefully integrated addition to working, tested code. Nothing from the original PRD v1.0 is removed or replaced. All changes are additive insertions.

Strategy	Status / Key behavior
Constant Dollar	Already implicit — made explicit. Zero new behavior.
Forgo Inflation Adjustment	New. Skip inflation raise after portfolio loss years.
Actual Spending Decline	New. Grow withdrawals at inflation minus real decline rate.
Guyton-Klinger Guardrails	New. Upper/lower rate guardrails with 10% adjustments.

Out of scope	Reason
Vanguard Dynamic Spending	Deferred to v1.3
Kitces/Tharp PoS Guardrails	Computationally prohibitive — deferred indefinitely
VPW / RMD / Dynamic SWR	Deferred to future releases

Table of Contents

Table of Contents	2
0. Critical Context — Read Before Writing Any Code	4
1. Worker Changes — simulation.worker.js	6
1.1 New input fields	6
1.2 Per-simulation state variables	6
1.3 Annual withdrawal calculation — replace existing Step 1	6
1.4 Cross-simulation accumulators	8
1.5 Income percentile path computation	9
1.6 Guyton-Klinger aggregate statistics	9
1.7 Updated results object	10
1.8 Worker test cases	11
2. Input Panel Changes — index.html + app.js	12
2.1 Strategy dropdown HTML	12
2.2 Minimum Annual Withdrawal HTML	12
2.3 Actual Spending parameter panel HTML	13
2.4 Guyton-Klinger parameter panel HTML	14
2.5 JavaScript — strategy handling functions (add to app.js)	16
2.6 Event listeners — add to existing listener block	19
2.7 Update inputs object + validation + reset	19
3. Results Display — Income Variability Report	21
3.1 HTML structure	21
3.2 Income fan chart JavaScript	22
3.3 Guardrail heatmap JavaScript	24
3.4 Income summary cards + G-K table + strategy callout	25
3.5 Wire up render + reset in existing results handler	28
4. Strategy Info Bubbles — Modal + CSS + Content	29
4.1 Modal HTML (add once near end of body)	29
4.2 Modal CSS (add to styles.css)	29
4.3 Strategy content object (add to app.js)	31
4.4 Modal JavaScript (add to app.js)	34
5. Final Verification Checklist	36
5.1 Worker	36
5.2 Input panel	36
5.3 Results display	36

5.4 Info bubbles 37

0. Critical Context — Read Before Writing Any Code

Before writing a single line of code, read these four files in full:

1. app.js — understand the inputs object construction, event listener patterns, and how results are currently rendered
2. simulation.worker.js — understand the annual calculation loop, bootstrap year drawing, and results object assembly
3. index.html — understand the existing section structure, HTML patterns, and element ID conventions
4. styles.css — understand CSS class names, input styling, section header patterns, and card styles

Warning: The golden rule: zero visual inconsistency. All new UI elements must be indistinguishable from existing elements in style, spacing, and behavior. Replace every [existing-*-class] placeholder with the actual class name from the existing HTML. Do not introduce new CSS class names unless absolutely necessary.

Files being modified:

File	What changes
simulation.worker.js	Annual withdrawal loop refactored to be strategy-aware
app.js	New input section, inputs object extended, new results section
index.html	New HTML for strategy inputs and Income Variability Report
styles.css	Minimal additions using existing patterns only

Files NOT changing:

simba_returns_data.json, render.yaml. Update README.md methodology section after all code changes are complete.

What is NOT changing in existing code:

- Bootstrap sampling logic
- Inflation tracking and cumulative inflation index
- Guaranteed income calculation (Social Security, pension)
- Portfolio return application and rebalancing logic
- Portfolio balance fan chart
- Success metrics (success rate, depletion statistics)
- Asset class reference table
- Progress indicator, disclaimer checkbox, reset functionality
- All existing input sections

Implementation sequence — follow this exact order:

1. Worker: implement constant_dollar branch — verify it preserves existing behavior exactly
2. Worker: implement forgo_inflation, actual_spending, guyton_klinger branches
3. Worker: add income percentile path computation
4. Worker: add gk_statistics computation
5. Worker: add new fields to results object
6. Input panel: insert Distribution Strategy section into left panel HTML
7. Input panel: wire up dropdown, parameter panels, validation, live previews
8. Input panel: add Minimum Annual Withdrawal input

9. Input panel: update inputs object sent to worker
10. Results display: add Income Variability Report section
11. Results display: render income fan chart
12. Results display: render guardrail heatmap (Guyton-Klinger only)
13. Results display: render income summary metric cards and G-K table
14. Info bubbles: add modal HTML, CSS, and JavaScript
15. End-to-end test: all four strategies, verify all outputs

1. Worker Changes — simulation.worker.js

1.1 New input fields

The worker receives an inputs object via postMessage. Add handling for these new fields at the top of the worker message handler. Default gracefully if absent for backward compatibility:

New fields — read at top of worker message handler

```
const distribution_strategy = inputs.distribution_strategy || 'constant_dollar';
const strategy_params = inputs.strategy_params || {};
const minimum_withdrawal_annual = inputs.minimum_withdrawal_annual || 0;

// Extract strategy params with safe defaults
const real_spending_decline_pct = strategy_params.real_spending_decline_pct || 2.0;
const upper_guardrail_pct = strategy_params.upper_guardrail_pct || 6.0;
const lower_guardrail_pct = strategy_params.lower_guardrail_pct || 4.0;
const gk_adjustment_pct = strategy_params.adjustment_pct || 10.0;
```

1.2 Per-simulation state variables

Add these variables at the top of the per-simulation loop. Reset for each simulation:

Per-simulation state — reset for each simulation

```
let current_withdrawal_nominal = null; // carried year to year
let prior_year_portfolio_return = null; // null on year 1
let initial_withdrawal_rate_pct = null; // for G-K
const gk_events = []; // guardrail events this simulation
const withdrawal_by_year = new Array(period_years).fill(0); // income path
```

1.3 Annual withdrawal calculation — replace existing Step 1

Note: Find the current code that determines the withdrawal amount for year t. Replace it entirely with the strategy-aware block below. Everything else in the loop is unchanged.

Year 1 initialization (runs only when t === 1):

Year 1 — identical for all strategies

```
if (t === 1) {
  const base_expense_today_dollars = getBucketExpense(1);
  const year1_expense_nominal = base_expense_today_dollars * inflation_index;
  current_withdrawal_nominal = year1_expense_nominal;
  initial_withdrawal_rate_pct = (current_withdrawal_nominal / portfolio_balance) * 100;
  withdrawal_by_year[0] = current_withdrawal_nominal;
}
```

Year t > 1 — strategy-aware calculation:

Strategy-aware withdrawal calculation — year $t > 1$

```

if (t > 1) {
  if (distribution_strategy === 'constant_dollar') {
    // Preserves v1.0 behavior exactly
    const target_today = getBucketExpense(t);
    current_withdrawal_nominal = target_today * inflation_index;
  } else if (distribution_strategy === 'forgo_inflation') {
    const target_today = getBucketExpense(t);
    const inflated_target = target_today * inflation_index;
    if (prior_year_portfolio_return < 0) {
      // Skip raise — hold flat, cap at inflated target for bucket transitions
      current_withdrawal_nominal = Math.min(current_withdrawal_nominal, inflated_target);
    } else {
      current_withdrawal_nominal = inflated_target;
    }
  } else if (distribution_strategy === 'actual_spending') {
    // bootstrap_inflation_pct = inflation from bootstrap year drawn for year t
    const net_growth_rate = (bootstrap_inflation_pct - real_spending_decline_pct) / 100;
    current_withdrawal_nominal = current_withdrawal_nominal * (1 + net_growth_rate);
    // Floor: never below 50% of year 1 withdrawal
    const withdrawal_floor = withdrawal_by_year[0] * 0.50;
    current_withdrawal_nominal = Math.max(current_withdrawal_nominal, withdrawal_floor);
    // Ceiling: never above 150% of current bucket inflation-adjusted target
    const target_nominal = getBucketExpense(t) * inflation_index;
    current_withdrawal_nominal = Math.min(current_withdrawal_nominal, target_nominal * 1.50);
  } else if (distribution_strategy === 'guyton_klinger') {
    // Rule 1: Inflation rule
    let proposed_withdrawal;
    if (prior_year_portfolio_return < 0) {
      proposed_withdrawal = current_withdrawal_nominal; // hold flat
    } else {
      proposed_withdrawal = current_withdrawal_nominal * (1 + bootstrap_inflation_pct / 100);
    }
    // Rule 2: Guardrail rate check
    const effective_rate_pct = (proposed_withdrawal / portfolio_balance) * 100;
    const years_remaining = period_years - t;
    const upper_active = years_remaining > 15; // final-15-year rule
    if (effective_rate_pct > upper_guardrail_pct && upper_active) {
      const old = proposed_withdrawal;
      proposed_withdrawal = proposed_withdrawal * (1 - gk_adjustment_pct / 100);
      gk_events.push({ year: t, type: 'upper', old_withdrawal: old,
        new_withdrawal: proposed_withdrawal,
        effective_rate_pct: effective_rate_pct });
    }
  }
}

```

```

} else if (effective_rate_pct < lower_guardrail_pct) {
  const old = proposed_withdrawal;
  proposed_withdrawal = proposed_withdrawal * (1 + gk_adjustment_pct / 100);
  gk_events.push({ year: t, type: 'lower', old_withdrawal: old,
    new_withdrawal: proposed_withdrawal,
    effective_rate_pct: effective_rate_pct });
}
current_withdrawal_nominal = proposed_withdrawal;
}

// Store this year's withdrawal
withdrawal_by_year[t - 1] = current_withdrawal_nominal;
}

```

Minimum withdrawal floor — apply after strategy block, before net withdrawal:

Minimum floor clamp — insert after strategy block, before net withdrawal calculation

```

if (minimum_withdrawal_annual > 0) {
  const minimum_nominal = inputs.inflation_adjustment_on
    ? minimum_withdrawal_annual * inflation_index
    : minimum_withdrawal_annual;

  const strategy_calculated = current_withdrawal_nominal;

  if (current_withdrawal_nominal < minimum_nominal) {
    current_withdrawal_nominal = minimum_nominal;
    sim_floor_binding_count++; // increment per-simulation floor binding counter
  }
}

// Update withdrawal_by_year after floor is applied
withdrawal_by_year[t - 1] = current_withdrawal_nominal;

```

After portfolio return is applied — update shared state:

Shared state update — end of year loop

```

// Add at end of year loop after bootstrap return is applied:
prior_year_portfolio_return = weighted_return;

```

1.4 Cross-simulation accumulators

Cross-simulation accumulators

```

// Declare BEFORE the simulation loop:
const withdrawal_all_sims = [];
const all_gk_events = [];
const floor_binding_counts = [];

// At END of each simulation (after year loop closes), push:
withdrawal_all_sims.push([...withdrawal_by_year]);

```

```

floor_binding_counts.push(sim_floor_binding_count); // reset sim_floor_binding_count=0 at sim start
if (distribution_strategy === 'guyton_klinger') {
  all_gk_events.push([...gk_events]);
}

```

1.5 Income percentile path computation

After all simulations complete, compute income percentile paths using the same methodology as portfolio balance paths:

Income percentile path computation

```

function computeIncomePercentilePaths(withdrawal_all_sims, period_years) {
  const percentiles = [10, 25, 50, 75, 90];
  const result = {};
  percentiles.forEach(p => { result['p' + p] = new Array(period_years).fill(0); });

  for (let year = 0; year < period_years; year++) {
    const year_values = withdrawal_all_sims
      .map(sim => sim[year]).sort((a, b) => a - b);
    percentiles.forEach(p => {
      const idx = Math.floor((p / 100) * year_values.length);
      result['p' + p][year] = year_values[Math.min(idx, year_values.length - 1)];
    });
  }
  return result;
}

const income_nominal_paths = computeIncomePercentilePaths(withdrawal_all_sims, period_years);

// Real versions: divide each year by median inflation index
// Use same median_inflation_path already computed for real portfolio paths
function computeRealIncomePaths(nominal_paths, median_inflation_indices) {
  const result = {};
  Object.keys(nominal_paths).forEach(key => {
    result[key] = nominal_paths[key].map((val, y) => val / (median_inflation_indices[y] || 1));
  });
  return result;
}

const income_real_paths = computeRealIncomePaths(income_nominal_paths, median_inflation_path);

```

1.6 Guyton-Klinger aggregate statistics

Guyton-Klinger aggregate statistics

```

function computeGKStatistics(all_gk_events, n_simulations, period_years) {
  if (!all_gk_events || all_gk_events.length === 0) return null;
  const avg = arr => arr.length ? arr.reduce((a, b) => a + b, 0) / arr.length : 0;
  const med = arr => {
    if (!arr.length) return null;

```

```

const s = [...arr].sort((a, b) => a - b);
const m = Math.floor(s.length / 2);
return s.length % 2 !== 0 ? s[m] : (s[m-1] + s[m]) / 2;
};

const sims_with_cut = all_gk_events.filter(e => e.some(ev => ev.type === 'upper')).length;
const sims_with_raise = all_gk_events.filter(e => e.some(ev => ev.type === 'lower')).length;
const cuts_per_sim = all_gk_events.map(e => e.filter(ev => ev.type === 'upper').length);
const raises_per_sim = all_gk_events.map(e => e.filter(ev => ev.type === 'lower').length);
const first_cut_years = all_gk_events
.map(e => e.find(ev => ev.type === 'upper')).filter(Boolean).map(ev => ev.year);

const pct_cuts_by_year = Array.from({ length: period_years }, (_, y) => {
const hits = all_gk_events
.filter(e => e.some(ev => ev.type === 'upper' && ev.year === y + 1)).length;
return (hits / n_simulations) * 100;
});

const pct_raises_by_year = Array.from({ length: period_years }, (_, y) => {
const hits = all_gk_events
.filter(e => e.some(ev => ev.type === 'lower' && ev.year === y + 1)).length;
return (hits / n_simulations) * 100;
});

return {
pct_sims_with_any_cut: (sims_with_cut / n_simulations) * 100,
pct_sims_with_any_raise: (sims_with_raise / n_simulations) * 100,
avg_cuts_per_sim: avg(cuts_per_sim),
avg_raises_per_sim: avg(raises_per_sim),
avg_year_of_first_cut: first_cut_years.length ? avg(first_cut_years) : null,
median_year_of_first_cut: first_cut_years.length ? med(first_cut_years) : null,
pct_cuts_by_year, pct_raises_by_year,
};
}

const gk_statistics = distribution_strategy === 'guyton_klinger'
? computeGKStatistics(all_gk_events, n_simulations, period_years)
: null;

```

1.7 Updated results object

Add these fields to the existing results object before postMessage:

Updated results object

```

{
// All existing fields unchanged ...

distribution_strategy: distribution_strategy,
year1_withdrawal_nominal: withdrawal_all_sims.length > 0 ? withdrawal_all_sims[0][0] : null,
year1_withdrawal_rate_pct: initial_withdrawal_rate_pct,

```

```

minimum_withdrawal_annual: minimum_withdrawal_annual,

income_percentile_paths: {
p10: income_nominal_paths.p10, p25: income_nominal_paths.p25,
p50: income_nominal_paths.p50, p75: income_nominal_paths.p75,
p90: income_nominal_paths.p90,
real_p10: income_real_paths.p10, real_p25: income_real_paths.p25,
real_p50: income_real_paths.p50, real_p75: income_real_paths.p75,
real_p90: income_real_paths.p90,
},

gk_statistics: gk_statistics,

floor_binding_percentiles: minimum_withdrawal_annual > 0
? computePercentileValues(floor_binding_counts, [10, 25, 50, 75, 90])
: null,
}

```

1.8 Worker test cases

Test scenario	Expected result
constant_dollar, no strategy field in inputs	Simulation completes. Success rate identical to pre-change results.
forgo_inflation, year 1 return -15%	Year 2 withdrawal = year 1 exactly. No inflation raise.
forgo_inflation, year 1 +8%, year 2 -10%	Year 2 = year1 x inflation. Year 3 = year 2 (no raise).
actual_spending, 2.5% inflation, 2.0% decline	Year 2 = year1 x 1.005.
actual_spending, 1.0% inflation, 2.0% decline	Net growth -1.0%. Floor check runs. No drop below 50% of year 1.
guyton_klinger, portfolio -20% year 1	Rule 1 holds flat. Rule 2 checks rate. Cut on held amount NOT inflated.
guyton_klinger, year 28 of 30, upper breach	years_remaining=2 <= 15. Cut NOT applied. No gk_event recorded.
guyton_klinger, year 28 of 30, lower breach	Raise IS applied. Lower guardrail has no final-15-year exception.
minimum_withdrawal_annual = \$80,000, strategy yields \$65,000	current_withdrawal_nominal set to \$80,000. floor_binding_count increments.
minimum_withdrawal_annual = 0	No floor logic runs. Behavior identical to pre-change.

2. Input Panel Changes — index.html + app.js

Note: Insert the Distribution Strategy section between the closing tag of Simulation Settings and the opening tag of Expense Schedule. Use the exact same section wrapper element and class names already used for other input sections. Replace every [existing-*-class] placeholder with the actual class name from the existing HTML.

2.1 Strategy dropdown HTML

Strategy dropdown — insert between Simulation Settings and Expense Schedule

```
<section class="[existing-section-class]">
<h3 class="[existing-section-header-class]">Distribution Strategy</h3>
<p class="[existing-section-description-class]">
Select how your annual portfolio withdrawal is calculated each year.
Constant Dollar treats your expense inputs as fixed real withdrawals.
Guardrail strategies dynamically adjust spending based on market performance.
</p>

<div class="[existing-input-group-class]">
<label for="distribution-strategy" class="[existing-label-class]">
Withdrawal Strategy
<button type="button" id="strategy-info-btn" class="info-icon-btn"
aria-label="Learn about this withdrawal strategy">i</button>
</label>
<select id="distribution-strategy" class="[existing-select-class]">
<option value="constant_dollar" selected>Constant Dollar (Bengen 4% rule)</option>
<option value="forgo_inflation">Forgo Inflation Adjustment (T. Rowe Price)</option>
<option value="actual_spending">Actual Spending Decline (EBRI / Blanchett)</option>
<option value="guyton_klinger">Guyton-Klinger Guardrails</option>
</select>
<p id="strategy-description" class="[existing-helper-text-class]">
Withdraws your target expense amount each year, adjusted upward for inflation.
Your portfolio absorbs all market gains and losses. This is the strategy
assumed in Bengen's original 4% rule research.
</p>
</div>
```

2.2 Minimum Annual Withdrawal HTML

Insert below the strategy dropdown/description, above the parameter panels. Always visible — not strategy-specific:

Minimum Annual Withdrawal input

```
<!-- Minimum Annual Withdrawal – always visible -->
<div class="[existing-input-group-class]">
<label for="minimum-withdrawal" class="[existing-label-class]">
Minimum Annual Withdrawal
</label>
```

```

<div class="[existing-currency-input-wrapper-class]">
<span class="[existing-currency-prefix-class]">$</span>
<input type="text" id="minimum-withdrawal"
class="[existing-currency-input-class]"
placeholder="0 (no floor)" value="">
</div>

<p class="[existing-helper-text-class]">
Optional. If your distribution strategy calculates a withdrawal below this
amount, this floor will be used instead. Represents your non-negotiable
baseline spending. Leave blank or $0 for no floor.
</p>

<div class="[existing-warning-box-class]" id="minimum-withdrawal-warning" hidden>
Warning: A high minimum withdrawal relative to your portfolio may accelerate
depletion. The simulation will deplete the portfolio rather than violate
the minimum floor.
</div>
</div>

```

2.3 Actual Spending parameter panel HTML

Actual Spending parameter panel

```

<!-- Actual Spending parameters – hidden by default -->
<div id="params-actual-spending" hidden>
<div class="[existing-input-group-class]">
<label for="real-spending-decline" class="[existing-label-class]">
Annual Real Spending Decline
</label>
<div class="[existing-input-suffix-wrapper-class]">
<input type="number" id="real-spending-decline"
min="0.5" max="3.0" step="any" value="2.0">
<span class="[existing-suffix-class]">% per year</span>
</div>
<p class="[existing-helper-text-class]">
Research from EBRI shows retirees naturally reduce real spending
approximately 2% per year. At 2.5% inflation and 2.0% decline,
withdrawals grow 0.5% per year in nominal terms.
</p>
<!-- Live preview -->
<div id="actual-spending-preview" class="[existing-info-box-class]">
<div class="preview-row">
<span>Net annual growth:</span><span id="preview-net-growth">0.5%</span>
</div>
<div class="preview-row">
<span>Year 1 withdrawal:</span><span id="preview-as-year1">—</span>

```

```

</div>
<div class="preview-row">
<span>Year 10 nominal:</span><span id="preview-as-year10-nominal">—</span>
</div>
<div class="preview-row">
<span>Year 10 real:</span><span id="preview-as-year10-real">—</span>
</div>
<div class="preview-row">
<span>Year 20 nominal:</span><span id="preview-as-year20-nominal">—</span>
</div>
<div class="preview-row">
<span>Year 20 real:</span><span id="preview-as-year20-real">—</span>
</div>
</div>
</div>
</div>
</div>

```

2.4 Guyton-Klinger parameter panel HTML

Guyton-Klinger parameter panel + section close

```

<!-- Guyton-Klinger parameters – hidden by default -->
<div id="params-guyton-klinger" hidden>
<!-- Upper guardrail -->
<div class="[existing-input-group-class]">
<label for="gk-upper-guardrail" class="[existing-label-class]">
Upper Guardrail (cut spending)
</label>
<div class="[existing-input-suffix-wrapper-class]">
<input type="number" id="gk-upper-guardrail"
min="4.0" max="8.0" step="any" value="6.0" placeholder="e.g. 6.0">
<span class="[existing-suffix-class]">%</span>
</div>
<p class="[existing-helper-text-class]">
Spending cut fires when effective withdrawal rate exceeds this.
Typical range 5.5%-6.5%. Enter any value between 4.0 and 8.0.
</p>
<p class="[existing-validation-error-class]" id="gk-upper-error" hidden></p>
</div>
<!-- Lower guardrail -->
<div class="[existing-input-group-class]">
<label for="gk-lower-guardrail" class="[existing-label-class]">
Lower Guardrail (raise spending)
</label>
<div class="[existing-input-suffix-wrapper-class]">

```

```

<input type="number" id="gk-lower-guardrail"
min="3.0" max="5.5" step="any" value="4.0" placeholder="e.g. 4.0">
<span class="[existing-suffix-class]">%</span>
</div>
<p class="[existing-helper-text-class]">
Spending raise fires when effective withdrawal rate falls below this.
Typical range 3.5%-4.5%. Enter any value between 3.0 and 5.5.
</p>
<p class="[existing-validation-error-class]" id="gk-lower-error" hidden></p>
</div>
<p class="[existing-validation-error-class]" id="gk-gap-error" hidden>
Upper guardrail must be at least 1.0% higher than lower guardrail.
</p>
<!-- Adjustment -->
<div class="[existing-input-group-class]">
<label for="gk-adjustment" class="[existing-label-class]">
Guardrail Adjustment
</label>
<div class="[existing-input-suffix-wrapper-class]">
<input type="number" id="gk-adjustment"
min="5" max="20" step="any" value="10" placeholder="e.g. 10">
<span class="[existing-suffix-class]">%</span>
</div>
<p class="[existing-helper-text-class]">
Percentage by which spending is cut or raised when guardrail breached.
Guyton-Klinger original research used 10%. Enter 5 to 20.
</p>
<p class="[existing-validation-error-class]" id="gk-adj-error" hidden></p>
</div>
<!-- Info note -->
<div class="[existing-info-box-class]">
<strong>Two rules apply each year in sequence:</strong><br>
Rule 1 – Inflation rule: skip raise if portfolio lost value last year.<br>
Rule 2 – Guardrail rule: check effective rate vs guardrails, adjust if breached.<br>
Rule 2 overrides Rule 1. Upper guardrail cut suspended in final 15 years.
</div>
<!-- Live rate preview -->
<div id="gk-rate-preview" class="[existing-info-box-class]">
<div class="preview-row">
<span>Year 1 effective rate:</span><span id="gk-preview-rate">--</span>
</div>
<div class="preview-row">
<span>Upper fires if portfolio falls to:</span>
<span id="gk-preview-upper-portfolio">--</span>

```

```

</div>
<div class="preview-row">
<span>Lower fires if portfolio grows to:</span>
<span id="gk-preview-lower-portfolio">--</span>
</div>
</div>
</div>
</div>
</section>
<!-- END DISTRIBUTION STRATEGY SECTION -->

```

2.5 JavaScript — strategy handling functions (add to app.js)

Strategy handling functions — add to app.js

```

const STRATEGY_DESCRIPTIONS = {
  constant_dollar: 'Withdraws your target expense amount each year, adjusted upward for inflation. ' +
    'Your portfolio absorbs all market gains and losses. This is the strategy ' +
    'assumed in Bengen's original 4% rule research.',
  forgo_inflation: 'Same as Constant Dollar, except the annual inflation raise is skipped in any ' +
    'year the portfolio lost value. A small, cumulative protection that compounds ' +
    'meaningfully over a 30+ year retirement.',
  actual_spending: 'Starts at your target expense level and increases withdrawals at a reduced ' +
    'rate — inflation minus your selected real spending decline rate. Reflects ' +
    'research showing retirees naturally spend less as they age.',
  guyton_klinger: 'Sets upper and lower withdrawal rate guardrails relative to your portfolio ' +
    'balance. When breached, spending adjusts up or down by the adjustment ' +
    'percentage. Accepts occasional income adjustments in exchange for higher ' +
    'normal-year withdrawals and improved portfolio longevity.',
};

function handleStrategyChange() {
  const strategy = document.getElementById('distribution-strategy').value;
  document.getElementById('strategy-description').textContent =
    STRATEGY_DESCRIPTIONS[strategy] || '';
  document.getElementById('params-actual-spending').hidden = strategy !== 'actual_spending';
  document.getElementById('params-guyton-klinger').hidden = strategy !== 'guyton_klinger';
  updateExpenseSectionNote(strategy);
  if (strategy === 'actual_spending') updateActualSpendingPreview();
  if (strategy === 'guyton_klinger') updateGKPreview();
  // Update modal if open
  if (!document.getElementById('strategy-info-modal').hidden) {
    buildModalContent(strategy);
  }
}

function updateExpenseSectionNote(strategy) {
  const note = document.getElementById('expense-section-note'); // adjust to actual ID

```

```

if (strategy === 'guyton_klinger' || strategy === 'actual_spending') {
  note.textContent = 'Enter your target expenses in today's dollars. The selected ' +
    'distribution strategy will determine the actual withdrawal each year – amounts ' +
    'may differ based on guardrail adjustments or spending decline rules.';
} else {
  note.textContent = ''; // restore original text from existing implementation
}
}

function updateActualSpendingPreview() {
  const decline = parseFloat(document.getElementById('real-spending-decline').value) || 2.0;
  const inflation = 2.5;
  const net_growth = inflation - decline;
  const year1 = getBucket1ExpenseValue();
  if (!year1 || year1 <= 0) return;
  const y10n = year1 * Math.pow(1 + net_growth/100, 9);
  const y10r = y10n / Math.pow(1 + inflation/100, 9);
  const y20n = year1 * Math.pow(1 + net_growth/100, 19);
  const y20r = y20n / Math.pow(1 + inflation/100, 19);
  document.getElementById('preview-net-growth').textContent =
    net_growth.toFixed(1) + '%' + (net_growth < 0 ? ' (real declining)' : '');
  document.getElementById('preview-as-year1').textContent = formatCurrency(year1);
  document.getElementById('preview-as-year10-nominal').textContent = formatCurrency(y10n);
  document.getElementById('preview-as-year10-real').textContent = formatCurrency(y10r);
  document.getElementById('preview-as-year20-nominal').textContent = formatCurrency(y20n);
  document.getElementById('preview-as-year20-real').textContent = formatCurrency(y20r);
}

function updateGKPreview() {
  const balance = getInitialBalance();
  const expense = getBucket1ExpenseValue();
  const upper = parseFloat(document.getElementById('gk-upper-guardrail').value) || 6.0;
  const lower = parseFloat(document.getElementById('gk-lower-guardrail').value) || 4.0;
  if (!balance || !expense) return;
  document.getElementById('gk-preview-rate').textContent =
    ((expense / balance) * 100).toFixed(1) + '%';
  document.getElementById('gk-preview-upper-portfolio').textContent =
    formatCurrency(expense / (upper / 100));
  document.getElementById('gk-preview-lower-portfolio').textContent =
    formatCurrency(expense / (lower / 100));
}

function validateGKInputs() {
  const upper = parseFloat(document.getElementById('gk-upper-guardrail').value);
  const lower = parseFloat(document.getElementById('gk-lower-guardrail').value);
  const adj = parseFloat(document.getElementById('gk-adjustment').value);
  let valid = true;

```

```

const ue = document.getElementById('gk-upper-error');
if (isNaN(upper) || upper < 4.0 || upper > 8.0) {
  ue.textContent = 'Upper guardrail must be between 4.0% and 8.0%';
  ue.hidden = false; valid = false;
} else { ue.hidden = true; }

const le = document.getElementById('gk-lower-error');
if (isNaN(lower) || lower < 3.0 || lower > 5.5) {
  le.textContent = 'Lower guardrail must be between 3.0% and 5.5%';
  le.hidden = false; valid = false;
} else { le.hidden = true; }

const ge = document.getElementById('gk-gap-error');
if (!isNaN(upper) && !isNaN(lower) && (upper - lower) < 1.0) {
  ge.hidden = false; valid = false;
} else { ge.hidden = true; }

const ae = document.getElementById('gk-adj-error');
if (isNaN(adj) || adj < 5 || adj > 20) {
  ae.textContent = 'Adjustment must be between 5% and 20%';
  ae.hidden = false; valid = false;
} else { ae.hidden = true; }

return valid;
}

function handleMinimumWithdrawalChange() {
  const raw = document.getElementById('minimum-withdrawal').value;
  const min_val = parseCurrencyInput(raw);
  const bucket1 = getBucket1ExpenseValue();
  if (min_val > 0) {
    document.getElementById('minimum-withdrawal').value = formatCurrencyInput(min_val);
  }

  document.getElementById('minimum-withdrawal-warning').hidden =
    !(bucket1 > 0 && min_val > bucket1 * 0.5);
  const strategy = document.getElementById('distribution-strategy').value;
  if (strategy === 'guyton_klinger') updateGKPreview();
}

function getStrategyInputs() {
  return {
    distribution_strategy: document.getElementById('distribution-strategy').value,
    minimum_withdrawal_annual: parseCurrencyInput(
      document.getElementById('minimum-withdrawal').value) || 0,
    strategy_params: {
      real_spending_decline_pct: parseFloat(
        document.getElementById('real-spending-decline').value) || 2.0,
      upper_guardrail_pct: parseFloat(
        document.getElementById('gk-upper-guardrail').value) || 6.0,
      lower_guardrail_pct: parseFloat(

```

```
document.getElementById('gk-lower-guardrail').value) || 4.0,
adjustment_pct: parseFloat(
document.getElementById('gk-adjustment').value) || 10.0,
},
};
}
```

2.6 Event listeners — add to existing listener block

Event listeners — add inside existing DOMContentLoaded handler

```
document.getElementById('distribution-strategy')
.addEventListener('change', handleStrategyChange);
document.getElementById('real-spending-decline')
.addEventListener('input', updateActualSpendingPreview);
document.getElementById('gk-upper-guardrail')
.addEventListener('input', updateGKPreview);
document.getElementById('gk-upper-guardrail')
.addEventListener('blur', validateGKInputs);
document.getElementById('gk-lower-guardrail')
.addEventListener('input', updateGKPreview);
document.getElementById('gk-lower-guardrail')
.addEventListener('blur', validateGKInputs);
document.getElementById('gk-adjustment')
.addEventListener('blur', validateGKInputs);
document.getElementById('minimum-withdrawal')
.addEventListener('blur', handleMinimumWithdrawalChange);

// Also call updateActualSpendingPreview() and updateGKPreview()
// inside the existing event handlers for initial balance and bucket 1 expense changes
```

2.7 Update inputs object + validation + reset

Inputs object / validation / reset updates

```
// In existing inputs object construction — add:
const strategy_inputs = getStrategyInputs();
Object.assign(inputs, strategy_inputs);

// In existing pre-simulation validation — add:
const strategy = document.getElementById('distribution-strategy').value;
if (strategy === 'guyton_klinger' && !validateGKInputs()) return;

// In existing reset handler — add:
document.getElementById('distribution-strategy').value = 'constant_dollar';
document.getElementById('real-spending-decline').value = '2.0';
document.getElementById('gk-upper-guardrail').value = '6.0';
document.getElementById('gk-lower-guardrail').value = '4.0';
document.getElementById('gk-adjustment').value = '10';
```

```
document.getElementById('minimum-withdrawal').value = '';  
document.getElementById('minimum-withdrawal-warning').hidden = true;  
handleStrategyChange();
```

3. Results Display — Income Variability Report

3.1 HTML structure

Insert below the existing results metrics table and above the Asset Class Reference Table:

Income Variability Report HTML

```
<section id="income-variability-report" class="[existing-results-section-class]" hidden>
<h2 class="[existing-results-section-header-class]">Income Variability Report</h2>
<p id="income-variability-subheader" class="[existing-section-description-class]"></p>

<!-- Income fan chart -->
<div class="[existing-chart-container-class]" style="height: 350px;">
<canvas id="income-fan-chart"></canvas>
</div>

<div class="[existing-chart-toggle-class]">
<button class="[existing-toggle-btn-class] [existing-active-class]"
data-chart="income" data-mode="nominal">Nominal</button>
<button class="[existing-toggle-btn-class]"
data-chart="income" data-mode="real">Real (inflation-adjusted)</button>
</div>

<!-- Guardrail heatmap — G-K only -->
<div id="guardrail-heatmap-container" hidden>
<h3 class="[existing-subsection-header-class]">Guardrail Activity by Simulation Year</h3>
<p class="[existing-helper-text-class]">
Red bars: % of simulations with spending cut (upper guardrail) each year.
Green bars: % with spending raise (lower guardrail). Taller red bars early
indicate years when poor markets most commonly triggered cuts.
</p>
<div class="[existing-chart-container-class]" style="height: 280px;">
<canvas id="guardrail-heatmap-chart"></canvas>
</div>
</div>

<!-- Income summary metric cards -->
<div id="income-summary-cards" class="[existing-metric-cards-container-class]"></div>

<!-- G-K income table — G-K only -->
<div id="gk-income-table-container" hidden>
<h3 class="[existing-subsection-header-class]">Income Statistics by Percentile</h3>
<div id="gk-income-table-wrapper"></div>
<p class="[existing-table-note-class]">
Total lifetime income is the sum of all annual withdrawals. Number of cuts/raises
reflects how frequently guardrail adjustments fired.
</p>
</div>
```

```
<!-- Strategy context callout -->
<div id="income-strategy-callout" class="[existing-info-box-class]"></div>
</section>
```

3.2 Income fan chart JavaScript

Income fan chart render function

```
let incomeFanChart = null;
let guardrailHeatmapChart = null;

function renderIncomeVariabilityReport(results) {
  const section = document.getElementById('income-variability-report');
  section.hidden = false;
  const strategy = results.distribution_strategy;
  const paths = results.income_percentile_paths;
  const start_age = results.inputs_summary.start_age;
  const period_years = results.inputs_summary.period_years;

  const subheaders = {
    constant_dollar: 'With Constant Dollar, income is fully predictable – the chart shows how ' +
      'inflation adjustments grow withdrawals over time.',
    forgo_inflation: 'With Forgo Inflation Adjustment, income is nearly stable – the chart shows ' +
      'occasional flat years when the market declines.',
    actual_spending: 'With Actual Spending Decline, income grows slowly in nominal terms and ' +
      'declines in real terms – matching how retirees actually spend.',
    guyton_klinger: 'With Guyton-Klinger Guardrails, income varies based on market performance. ' +
      'The chart shows the full range of annual withdrawal outcomes.',
  };

  document.getElementById('income-variability-subheader').textContent =
    subheaders[strategy] || '';

  const labels = Array.from({ length: period_years }, (_, i) => start_age + i + 1);

  renderIncomeFanChart(results, labels, 'nominal');
  document.querySelectorAll('[data-chart="income"]').forEach(btn => {
    btn.addEventListener('click', () => {
      document.querySelectorAll('[data-chart="income"]').forEach(b =>
        b.classList.remove('[existing-active-class]'));
      btn.classList.add('[existing-active-class]');
      renderIncomeFanChart(results, labels, btn.dataset.mode);
    });
  });

  const hc = document.getElementById('guardrail-heatmap-container');
  hc.hidden = strategy !== 'guyton_klinger';
  if (strategy === 'guyton_klinger' && results.gk_statistics) {
    renderGuardrailHeatmap(results.gk_statistics, labels);
  }
}
```

```
renderIncomeSummaryCards(results, labels);

const gc = document.getElementById('gk-income-table-container');
gc.hidden = strategy !== 'guyton_klinger';
if (strategy === 'guyton_klinger') renderGKIncomeTable(results);

renderStrategyCallout(strategy, results);
}

function renderIncomeFanChart(results, labels, mode) {
  if (incomeFanChart) { incomeFanChart.destroy(); incomeFanChart = null; }
  const ctx = document.getElementById('income-fan-chart').getContext('2d');
  const strategy = results.distribution_strategy;
  const paths = results.income_percentile_paths;
  const show_bands = strategy === 'actual_spending' || strategy === 'guyton_klinger';
  const p = key => mode === 'real' ? paths['real_' + key] : paths[key];
  const year1 = results.year1_withdrawal_nominal;
  const datasets = [];

  if (show_bands) {
    datasets.push({ label: '10th-90th percentile', data: p('p90'),
      backgroundColor: 'rgba(59,130,246,0.15)', borderColor: 'transparent',
      fill: '1', pointRadius: 0, tension: 0.3 });
    datasets.push({ label: '_p10', data: p('p10'),
      backgroundColor: 'rgba(59,130,246,0.15)', borderColor: 'transparent',
      fill: false, pointRadius: 0, tension: 0.3 });
    datasets.push({ label: '25th-75th percentile', data: p('p75'),
      backgroundColor: 'rgba(59,130,246,0.25)', borderColor: 'transparent',
      fill: '3', pointRadius: 0, tension: 0.3 });
    datasets.push({ label: '_p25', data: p('p25'),
      backgroundColor: 'rgba(59,130,246,0.25)', borderColor: 'transparent',
      fill: false, pointRadius: 0, tension: 0.3 });
  }

  datasets.push({ label: 'Median (50th)', data: p('p50'),
    borderColor: '#3b82f6', backgroundColor: 'transparent',
    borderWidth: 2.5, fill: false, pointRadius: 0, tension: 0.3 });
  datasets.push({ label: 'Year 1 target',
    data: new Array(labels.length).fill(year1),
    borderColor: '#f59e0b', backgroundColor: 'transparent',
    borderWidth: 1, borderDash: [6,4], fill: false, pointRadius: 0 });

  if (strategy === 'guyton_klinger') {
    const adj = parseFloat(document.getElementById('gk-adjustment').value) || 10;
    datasets.push({ label: 'First cut level',
      data: new Array(labels.length).fill(year1 * (1 - adj/100)),
      borderColor: '#f59e0b', backgroundColor: 'transparent',
      borderWidth: 1, borderDash: [3,3], fill: false, pointRadius: 0 });
  }
}
```

```

incomeFanChart = new Chart(ctx, {
  type: 'line', data: { labels, datasets },
  options: {
    responsive: true, maintainAspectRatio: false,
    interaction: { mode: 'index', intersect: false },
    plugins: {
      legend: { position: 'bottom',
        labels: { filter: item => !item.text.startsWith('_') } },
      title: { display: true,
        text: 'Projected Annual Income – ' +
          results.inputs_summary.n_simulations.toLocaleString() + ' Simulations' },
      tooltip: { callbacks: {
        label: ctx => ctx.dataset.label.startsWith('_') ? null
          : ctx.dataset.label + ': ' + formatCurrency(ctx.parsed.y) } },
    },
    scales: {
      x: { title: { display: true, text: 'Age' },
        ticks: { callback: (v,i) => i % 5 === 0 ? labels[i] : '', maxRotation: 0 } },
      y: { min: 0,
        title: { display: true, text: mode === 'real'
          ? 'Annual Withdrawal (Real)' : 'Annual Withdrawal (Nominal)' },
        ticks: { callback: val => formatCurrencyAbbreviated(val) } },
    },
  },
});

```

3.3 Guardrail heatmap JavaScript

Guardrail heatmap render function

```

function renderGuardrailHeatmap(gk_statistics, labels) {
  if (guardrailHeatmapChart) { guardrailHeatmapChart.destroy(); guardrailHeatmapChart = null; }
  const ctx = document.getElementById('guardrail-heatmap-chart').getContext('2d');
  guardrailHeatmapChart = new Chart(ctx, {
    type: 'bar',
    data: { labels, datasets: [
      { label: 'Upper guardrail hit (spending cut)',
        data: gk_statistics.pct_cuts_by_year,
        backgroundColor: 'rgba(239,68,68,0.7)', borderColor: '#ef4444', borderWidth: 1 },
      { label: 'Lower guardrail hit (spending raise)',
        data: gk_statistics.pct_raises_by_year,
        backgroundColor: 'rgba(16,185,129,0.7)', borderColor: '#10b981', borderWidth: 1 },
    ]},
    options: {

```

```

responsive: true, maintainAspectRatio: false,
plugins: {
  legend: { position: 'bottom' },
  title: { display: true, text: 'Guardrail Activity by Simulation Year' },
  tooltip: { callbacks: {
    label: ctx => ctx.dataset.label + ': ' + ctx.parsed.y.toFixed(1) + '% of simulations'
  }},
},
scales: {
  x: { title: { display: true, text: 'Simulation Year' },
    ticks: { callback: (v,i) => i % 5 === 0 ? labels[i] : '', maxRotation: 0 } },
  y: { min: 0, max: 100,
    title: { display: true, text: '% of Simulations' },
    ticks: { callback: val => val + '%' } },
},
});
}

```

3.4 Income summary cards + G-K table + strategy callout

Summary cards, G-K table, and strategy callout

```

function renderIncomeSummaryCards(results, labels) {
  const container = document.getElementById('income-summary-cards');
  const strategy = results.distribution_strategy;
  const paths = results.income_percentile_paths;
  const period_years = results.inputs_summary.period_years;
  const start_age = results.inputs_summary.start_age;
  const year1 = results.year1_withdrawal_nominal;
  const end_age = start_age + period_years;
  const mid_year = Math.min(14, period_years - 1);
  const mid_age = start_age + mid_year + 1;

  let card4_label, card4_value, card4_sub;
  if (strategy === 'constant_dollar') {
    card4_label = 'Real income – final year';
    card4_value = formatCurrency(paths.real_p50[period_years - 1]);
    card4_sub = 'Median – purchasing power in today's dollars';
  } else if (strategy === 'forgo_inflation') {
    card4_label = 'Median final year income';
    card4_value = formatCurrency(paths.p50[period_years - 1]);
    card4_sub = 'Nominal – 50th percentile';
  } else if (strategy === 'actual_spending') {
    let crossover = null;
    for (let y = 0; y < period_years; y++) {

```

```

if (paths.p50[y] >= year1 * Math.pow(1.025, y)) {
  crossover = start_age + y + 1; break;
}
}
card4_label = 'Constant Dollar crossover';
card4_value = crossover ? 'Age ' + crossover : 'Beyond simulation';
card4_sub = 'Age when nominal income matches Constant Dollar equivalent';
} else if (strategy === 'guyton_klinger') {
  const gk = results.gk_statistics;
  card4_label = 'Simulations with at least 1 cut';
  card4_value = gk ? gk.pct_sims_with_any_cut.toFixed(1) + '%' : '-';
  card4_sub = 'Upper guardrail triggered at least once';
}

container.innerHTML = `
<div class="[existing-metric-card-class]">
<div class="[existing-card-label-class]">Year 1 Withdrawal</div>
<div class="[existing-card-value-class]">\${formatCurrency(year1)}</div>
<div class="[existing-card-sub-class]">
\${results.year1_withdrawal_rate_pct
? results.year1_withdrawal_rate_pct.toFixed(1) + '% of starting portfolio' : ''}
</div>
</div>

<div class="[existing-metric-card-class]">
<div class="[existing-card-label-class]">Median Final Year Withdrawal</div>
<div class="[existing-card-value-class]">
\${formatCurrency(paths.p50[period_years - 1])}
</div>
<div class="[existing-card-sub-class]">Age \${end_age} – 50th percentile (nominal)</div>
</div>

<div class="[existing-metric-card-class]">
<div class="[existing-card-label-class]">Income Range at Age \${mid_age}</div>
<div class="[existing-card-value-class]">
\${formatCurrency(paths.p10[mid_year])} – \${formatCurrency(paths.p90[mid_year])}
</div>
<div class="[existing-card-sub-class]">10th to 90th percentile</div>
</div>

<div class="[existing-metric-card-class]">
<div class="[existing-card-label-class]">\${card4_label}</div>
<div class="[existing-card-value-class]">\${card4_value}</div>
<div class="[existing-card-sub-class]">\${card4_sub}</div>
</div>
`;
}

function renderGKIncomeTable(results) {

```

```

if (!results.gk_statistics) return;
const gk = results.gk_statistics;
const paths = results.income_percentile_paths;
const sa = results.inputs_summary.start_age;
const total = key => paths[key].reduce((a, b) => a + b, 0);

document.getElementById('gk-income-table-wrapper').innerHTML = `
<table class="[existing-results-table-class]">
<thead><tr><th>Metric</th><th>Value</th></tr></thead>
<tbody>
<tr><td>Simulations with at least one spending cut</td>
<td>\${gk.pct_sims_with_any_cut.toFixed(1)}%</td></tr>
<tr><td>Simulations with at least one spending raise</td>
<td>\${gk.pct_sims_with_any_raise.toFixed(1)}%</td></tr>
<tr><td>Average spending cuts per simulation</td>
<td>\${gk.avg_cuts_per_sim.toFixed(1)}</td></tr>
<tr><td>Average spending raises per simulation</td>
<td>\${gk.avg_raises_per_sim.toFixed(1)}</td></tr>
<tr><td>Average year of first spending cut</td>
<td>\${gk.avg_year_of_first_cut !== null
? 'Year ' + gk.avg_year_of_first_cut.toFixed(1) +
' (Age ' + (sa + gk.avg_year_of_first_cut).toFixed(0) + ' )'
: 'No cuts occurred'}</td></tr>
<tr><td>Median year of first spending cut</td>
<td>\${gk.median_year_of_first_cut !== null
? 'Year ' + gk.median_year_of_first_cut +
' (Age ' + (sa + gk.median_year_of_first_cut) + ' )'
: 'No cuts occurred'}</td></tr>
<tr><td>Median total lifetime income (nominal)</td>
<td>\${formatCurrency(total('p50'))}</td></tr>
<tr><td>Median total lifetime income (real)</td>
<td>\${formatCurrency(total('real_p50'))}</td></tr>
</tbody>
</table>
`;
}

function renderStrategyCallout(strategy, results) {
const callout = document.getElementById('income-strategy-callout');
const gk = results.gk_statistics;
const texts = {
  constant_dollar:
    'With Constant Dollar, income is fully predictable. Portfolio success rate ' +
    'reflects whether the fixed withdrawal schedule depleted the portfolio.',
  forgo_inflation:
    'With Forgo Inflation Adjustment, income is nearly stable. Each skipped raise ' +

```

```

'stays invested and compounds forward – preserving meaningful portfolio value ' +
'over a 35-year retirement without any active decision-making.',
actual_spending:
'With Actual Spending Decline, lower withdrawals in later years preserve the ' +
'portfolio. This strategy typically produces a higher success rate than Constant ' +
'Dollar at the same starting balance, while delivering more income in the ' +
'active early years of retirement.',
guyton_klinger: gk
? 'Guardrail strategies trade income predictability for portfolio longevity. In ' +
'these simulations, ' + gk.pct_sims_with_any_cut.toFixed(1) + '% of scenarios ' +
'triggered at least one spending cut. Simulations where the upper guardrail fires ' +
'frequently tend to have higher portfolio survival rates – the strategy is working ' +
'as designed.' : '',
};
let text = texts[strategy] || '';
if (results.minimum_withdrawal_annual > 0) {
const floor_p50 = results.floor_binding_percentiles
? results.floor_binding_percentiles[2] : 0;
text += ' A minimum annual withdrawal floor of ' +
formatCurrency(results.minimum_withdrawal_annual) + ' was active. ' +
'In the median scenario the floor was binding in ' + floor_p50 +
' of ' + results.inputs_summary.period_years + ' years.';
}
callout.textContent = text;
}

```

3.5 Wire up render + reset in existing results handler

Wire up render and reset

```

// At END of existing displayResults() function – add:
renderIncomeVariabilityReport(results);

// At START of new simulation run (where results are cleared) – add:
document.getElementById('income-variability-report').hidden = true;
if (incomeFanChart) { incomeFanChart.destroy(); incomeFanChart = null; }
if (guardrailHeatmapChart) { guardrailHeatmapChart.destroy(); guardrailHeatmapChart = null; }

```

4. Strategy Info Bubbles — Modal + CSS + Content

An info icon (i) appears inline with the "Withdrawal Strategy" label. When clicked, it opens a modal overlay with a plain-English description of the selected strategy. Pure UI — no simulation engine changes. The label HTML was already shown in Section 2.1 above (the strategy-info-btn button).

4.1 Modal HTML (add once near end of body)

Modal HTML — add near end of body

```
<div id="strategy-info-modal" class="modal-overlay" hidden
role="dialog" aria-modal="true" aria-labelledby="modal-title">
<div class="modal-panel">
<div class="modal-header">
<h2 id="modal-title" class="modal-title"></h2>
<button type="button" id="modal-close-btn"
class="modal-close-btn" aria-label="Close">x</button>
</div>
<div class="modal-body" id="modal-body"></div>
<div class="modal-footer">
<button type="button" id="modal-close-footer-btn"
class="[existing-secondary-btn-class]">Close</button>
</div>
</div>
</div>
```

4.2 Modal CSS (add to styles.css)

Modal CSS — add to styles.css

```
.info-icon-btn {
display: inline-flex; align-items: center; justify-content: center;
width: 18px; height: 18px; margin-left: 6px;
background: transparent; border: 1.5px solid var(--color-primary-blue);
border-radius: 50%; color: var(--color-primary-blue);
font-size: 11px; font-weight: 700; cursor: pointer;
vertical-align: middle; line-height: 1; padding: 0;
transition: background 0.15s, color 0.15s;
}
.info-icon-btn:hover {
background: var(--color-primary-blue); color: var(--color-text-primary);
}
.modal-overlay {
position: fixed; inset: 0; background: rgba(0,0,0,0.65);
display: flex; align-items: center; justify-content: center; z-index: 1000;
}
.modal-overlay[hidden] { display: none; }
```

```
.modal-panel {
background: var(--color-surface); border: 1px solid var(--color-border);
border-radius: 12px; width: 580px; max-width: 92vw; max-height: 82vh;
display: flex; flex-direction: column; overflow: hidden;
}
.modal-header {
display: flex; align-items: center; justify-content: space-between;
padding: 20px 24px 16px; border-bottom: 1px solid var(--color-border); flex-shrink: 0;
}
.modal-title { font-size: 16px; font-weight: 600; color: var(--color-text-primary); margin: 0; }
.modal-close-btn {
background: transparent; border: none; color: var(--color-text-secondary);
font-size: 18px; cursor: pointer; padding: 4px 8px; border-radius: 4px;
}
.modal-close-btn:hover { color: var(--color-text-primary); background: var(--color-surface-light); }
.modal-body {
padding: 20px 24px; overflow-y: auto; flex: 1;
color: var(--color-text-secondary); font-size: 14px; line-height: 1.6;
}
.modal-body h3 {
font-size: 13px; font-weight: 600; color: var(--color-text-primary);
margin: 16px 0 6px; text-transform: uppercase; letter-spacing: 0.04em;
}
.modal-body h3:first-child { margin-top: 0; }
.modal-body p { margin: 0 0 12px; }
.modal-scenario {
background: var(--color-surface-light);
border-left: 3px solid var(--color-primary-blue);
border-radius: 0 6px 6px 0; padding: 10px 14px; margin: 8px 0; font-size: 13px;
}
.modal-scenario.steady { border-left-color: var(--color-success); }
.modal-scenario.bear { border-left-color: var(--color-danger); }
.modal-scenario.bull { border-left-color: var(--color-primary-blue); }
.modal-scenario-label {
font-weight: 600; color: var(--color-text-primary); display: block;
margin-bottom: 4px; font-size: 12px; text-transform: uppercase; letter-spacing: 0.03em;
}
.modal-warning {
background: rgba(245,158,11,0.1); border: 1px solid var(--color-warning);
border-radius: 6px; padding: 10px 14px; font-size: 13px; margin-top: 12px;
}
.modal-footer {
padding: 14px 24px; border-top: 1px solid var(--color-border);
display: flex; justify-content: flex-end; flex-shrink: 0;
```

```
}

```

4.3 Strategy content object (add to app.js)

This object drives all modal content. Place near top of app.js or in a separate strategy-content.js file if app.js is already large:

STRATEGY_INFO_CONTENT object — add to app.js

```
const STRATEGY_INFO_CONTENT = {
  constant_dollar: {
    title: 'Constant Dollar – Bengen 4% Rule',
    sections: [
      { heading: 'How it works',
        body: 'Withdraw a fixed dollar amount in year 1, then increase that exact dollar ' +
        'amount by inflation each year – no exceptions. The portfolio absorbs all ' +
        'market gains and losses silently. No decisions required after initial setup. ' +
        'This is the strategy assumed in William Bengen's original 1994 research.' },
      { heading: 'Best for',
        body: 'Retirees who want total spending certainty and have a pension or Social ' +
        'Security that covers baseline needs. Comfortable leaving money on the table ' +
        'in good markets. Focused on preserving wealth for heirs.' },
      { heading: 'What this feels like',
        scenarios: [
          { type: 'steady', label: 'Steady market',
            text: 'Calm and predictable. Income rises slightly each year with inflation. ' +
            'The couple never thinks about their strategy – it just works.' },
          { type: 'bear', label: 'Bear market',
            text: 'Income stays the same – no panic, no decisions. But the portfolio ' +
            'quietly absorbs the full loss. If losses continue, the effective ' +
            'withdrawal rate on the shrinking portfolio climbs dangerously.' },
          { type: 'bull', label: 'Bull market',
            text: 'Income stays the same even though the portfolio jumped. Good for ' +
            'building a bequest; less satisfying for those who want to enjoy a ' +
            'strong year in their spending.' },
        ] },
      { heading: 'Key risk',
        body: 'Sequence-of-returns risk. A major bear market early in retirement forces ' +
        'the same high withdrawal from a much smaller portfolio, dramatically ' +
        'increasing the risk of depletion if losses persist.' },
    ],
  },
  forgo_inflation: {
    title: 'Forgo Inflation Adjustment – T. Rowe Price Method',
    sections: [
      { heading: 'How it works',
        body: 'Behave exactly like Constant Dollar – except if the portfolio lost value ' +
```

```

'this year, hold spending flat next year instead of applying the inflation ' +
'raise. One simple rule change with meaningful long-term compounding ' +
'protection. The skipped raise is permanent – it does not catch up.' },
{ heading: 'Best for',
body: 'Retirees who want a paycheck-like income stream and are comfortable ' +
'skipping inflation raises after a down year. Identical to Constant Dollar ' +
'in positive market years – the protection only activates when needed.' },
{ heading: 'What this feels like',
scenarios: [
{ type: 'steady', label: 'Steady market',
text: 'Identical to Constant Dollar – income rises with inflation. ' +
'The protection mechanism never fires in a positive year.' },
{ type: 'bear', label: 'Bear market',
text: 'Income holds flat next year instead of rising with inflation. ' +
'A modest protection – but the skipped raise compounds forward ' +
'through every remaining year, preserving meaningful portfolio value.' },
{ type: 'bull', label: 'Bull market',
text: 'Identical to Constant Dollar – income rises with inflation. ' +
'The strategy only activates after losses.' },
] },
{ heading: 'Key advantage over Constant Dollar',
body: 'Each skipped raise stays invested and compounds forward. Research shows ' +
'this small, repeated protection meaningfully extends portfolio longevity ' +
'over a 35-year retirement without requiring any active decision-making.' },
],
},

actual_spending: {
title: 'Actual Spending Decline – EBRI / Blanchett Method',
sections: [
{ heading: 'How it works',
body: 'Start at your target expense level and increase withdrawals at a reduced ' +
'rate – inflation minus your selected real spending decline rate. At the ' +
'default 2% real decline with 2.5% inflation, withdrawals grow only 0.5% ' +
'per year in nominal terms and actually decline in real terms. This reflects ' +
'decades of research showing retirees naturally spend less as they age.' },
{ heading: 'The research behind it',
body: 'EBRI found that inflation-adjusted household spending declines approximately ' +
'19% from age 65 to 75, 34% from 65 to 85, and 52% from 65 to 95. David ' +
'Blanchett modeled this as roughly 2% real decline per year. Morningstar ' +
'2025 research confirms this strategy supports a 5.0% starting withdrawal ' +
'rate at 90% success over 30 years.' },
{ heading: 'Best for',
body: 'Retirees who want to spend more in the active early years of retirement, ' +
'with the expectation that spending will naturally decline as they age. ' +

```

```

'Requires honest self-assessment – the strategy assumes you genuinely will ' +
'spend less at 80 than at 65.' },
{ heading: 'What this feels like',
scenarios: [
{ type: 'steady', label: 'Steady market',
text: 'Income ticks up very slightly each year. But the couple starts ' +
'retirement earning significantly more than Constant Dollar in the ' +
'active go-go years. Lifestyle stays essentially flat in nominal terms.' },
{ type: 'bear', label: 'Bear market',
text: 'Income continues growing at the net rate despite the bear year – same ' +
'as steady market. No cut, no decision. Same sequence-of-returns risk ' +
'as Constant Dollar applies.' },
{ type: 'bull', label: 'Bull market',
text: 'Income grows at the same modest net rate regardless of market strength. ' +
'The portfolio surges but the couple captures none of it. Gains ' +
'accumulate for longevity protection.' },
] },
{ heading: 'Key trade-off',
body: 'Constant Dollar eventually catches up in nominal income – typically around ' +
'age 73. But by then spending research shows retirees' actual needs have ' +
'declined. The strategy is designed around how people actually live, not ' +
'an inflation formula.' },
],
},
guyton_klinger: {
title: 'Guyton-Klinger Guardrails',
sections: [
{ heading: 'How it works',
body: 'Two rules apply in sequence each year. Rule 1 – Inflation rule: if the ' +
'portfolio lost value last year, skip the inflation raise and hold spending ' +
'flat. Rule 2 – Guardrail rule: calculate your effective withdrawal rate ' +
'(annual withdrawal / current portfolio balance). If it exceeds the upper ' +
'guardrail, cut spending by the adjustment %. If it falls below the lower ' +
'guardrail, raise spending by the adjustment %. Rule 2 overrides Rule 1. ' +
'The upper guardrail cut is suspended in the final 15 years.' },
{ heading: 'Best for',
body: 'Retirees who want to maximize what they spend during their lifetime and ' +
'are willing to accept occasional income adjustments in exchange for a ' +
'higher starting withdrawal rate. Works best when Social Security or ' +
'pension income covers essential expenses – so guardrail cuts affect ' +
'discretionary spending rather than basic needs.' },
{ heading: 'What this feels like',
scenarios: [
{ type: 'steady', label: 'Steady market',

```

```

text: 'Income rises with inflation – rate stays within the guardrail band. ' +
'The couple earns significantly more than Constant Dollar with no ' +
'decisions required.' },
{ type: 'bear', label: 'Bear market',
text: 'Both rules may activate. First the inflation raise is skipped, then ' +
'the guardrail rate check fires a spending cut. A meaningful income ' +
'reduction – but the couple still typically earns more than Constant ' +
'Dollar even after the cut. Higher normal income in exchange for ' +
'accepting real cuts when markets turn.' },
{ type: 'bull', label: 'Bull market',
text: 'Income rises with inflation. In a sufficiently strong year the lower ' +
'guardrail may fire, triggering a spending raise. Strong markets ' +
'accumulate quietly until the lower threshold is reached.' },
] },
{ heading: 'Key advantage over Constant Dollar',
body: 'Morningstar 2025 research shows Guyton-Klinger supports a 5.2% starting ' +
'safe withdrawal rate versus 3.9% for Constant Dollar at the same 90% ' +
'probability of success over 30 years. Lifetime spending is higher in the ' +
'median scenario despite occasional cuts.' },
{ heading: 'Key risk – important',
body: 'In severe or prolonged bear markets, the upper guardrail may fire ' +
'repeatedly, producing meaningful cumulative income cuts. Research shows ' +
'this strategy can require cuts exceeding 40% of original income in the ' +
'worst historical sequences. A minimum withdrawal floor helps protect ' +
'against this.',
is_warning: true },
],
},
};

```

4.4 Modal JavaScript (add to app.js)

Modal JavaScript — add to app.js

```

function buildModalContent(strategy) {
const content = STRATEGY_INFO_CONTENT[strategy];
if (!content) return;
document.getElementById('modal-title').textContent = content.title;
let html = '';
content.sections.forEach(section => {
html += '<h3>' + section.heading + '</h3>';
if (section.body) {
const cls = section.is_warning ? 'modal-warning' : '';
html += '<p class="' + cls + '>' + section.body + '</p>';
}
}
}

```

```
if (section.scenarios) {
  section.scenarios.forEach(sc => {
    html += '<div class="modal-scenario ' + sc.type + '">' +
    '<span class="modal-scenario-label">' + sc.label + '</span>' +
    sc.text + '</div>';
  });
}

document.getElementById('modal-body').innerHTML = html;
}

function openStrategyModal() {
  const strategy = document.getElementById('distribution-strategy').value;
  buildModalContent(strategy);
  document.getElementById('strategy-info-modal').hidden = false;
  document.getElementById('modal-close-btn').focus();
  document.addEventListener('keydown', handleModalKeydown);
}

function closeStrategyModal() {
  document.getElementById('strategy-info-modal').hidden = true;
  document.removeEventListener('keydown', handleModalKeydown);
  document.getElementById('strategy-info-btn').focus();
}

function handleModalKeydown(e) {
  if (e.key === 'Escape') closeStrategyModal();
}

// Wire up – add to existing listener block:
document.getElementById('strategy-info-btn')
  .addEventListener('click', openStrategyModal);
document.getElementById('modal-close-btn')
  .addEventListener('click', closeStrategyModal);
document.getElementById('modal-close-footer-btn')
  .addEventListener('click', closeStrategyModal);
document.getElementById('strategy-info-modal')
  .addEventListener('click', e => {
    if (e.target === document.getElementById('strategy-info-modal')) closeStrategyModal();
  });

// Add inside handleStrategyChange() – update modal if already open:
// if (!document.getElementById('strategy-info-modal').hidden) {
//   buildModalContent(document.getElementById('distribution-strategy').value);
// }
```

5. Final Verification Checklist

5.1 Worker

- ☐ Simulation runs without errors with no `distribution_strategy` in inputs
- ☐ Constant Dollar results are byte-for-byte identical to pre-change results
- ☐ `income_percentile_paths` in results object — 10 arrays of length = `period_years`
- ☐ `gk_statistics` is null for non-G-K strategies
- ☐ `gk_statistics` populated with sensible values for Guyton-Klinger
- ☐ `forgo_inflation`: year 2 withdrawal equals year 1 exactly when prior return was negative
- ☐ `actual_spending`: year 2 = year 1 x 1.005 at 2.5% inflation / 2.0% decline
- ☐ `actual_spending`: floor check prevents drop below 50% of year 1
- ☐ G-K: cut applied to held (not raised) withdrawal amount after negative year
- ☐ G-K final-15-year rule: no upper cut when `years_remaining` <= 15
- ☐ G-K lower guardrail: raise applied regardless of years remaining
- ☐ Minimum floor: `floor_binding_count` increments when floor overrides strategy
- ☐ Minimum floor: inflates with `inflation_index` when `inflation_adjustment_on` is true
- ☐ Minimum floor: stays nominal when `inflation_adjustment_on` is false
- ☐ `floor_binding_percentiles` is null when `minimum_withdrawal_annual` = 0

5.2 Input panel

- ☐ Distribution Strategy section appears between Simulation Settings and Expense Schedule
- ☐ Section visually consistent with existing sections — no new CSS class names introduced
- ☐ Strategy dropdown changes show/hide correct parameter panels
- ☐ Description callout text updates correctly on dropdown change
- ☐ Actual spending preview updates when decline rate or bucket 1 expense changes
- ☐ G-K preview updates when guardrail values or initial balance changes
- ☐ G-K validation blocks simulation when inputs are invalid
- ☐ G-K cross-field gap error shows when upper - lower < 1.0%
- ☐ Minimum withdrawal warning shows when floor > 50% of bucket 1 expense
- ☐ Minimum withdrawal formats with commas on blur
- ☐ Minimum withdrawal = blank or \$0 means no floor applied
- ☐ Run Simulation button remains disabled when G-K inputs are invalid
- ☐ Reset restores all new inputs to defaults
- ☐ Expense section note updates for guardrail strategies
- ☐ Info icon (i) appears inline with Withdrawal Strategy label

5.3 Results display

- ☐ Income Variability Report appears below results metrics table

- ☐ Report is hidden while simulation runs, visible after results arrive
- ☐ Subheader text matches selected strategy
- ☐ Income fan chart renders for all four strategies
- ☐ Nominal/Real toggle updates income fan chart
- ☐ Year 1 target reference line shown (gold dashed)
- ☐ G-K first cut level reference line shown (amber dashed) — G-K only
- ☐ For constant_dollar / forgo_inflation: no filled bands shown (near-deterministic)
- ☐ Guardrail heatmap appears only for Guyton-Klinger
- ☐ Guardrail heatmap shows sensible red/green bar distribution
- ☐ Income summary cards render for all four strategies
- ☐ Card 4 content is strategy-specific and correct
- ☐ G-K income table appears only for Guyton-Klinger
- ☐ Strategy callout text is correct for each strategy
- ☐ Minimum floor callout note appears in strategy callout when floor > 0
- ☐ Floor binding row in results metrics table shows only when floor > 0
- ☐ Charts destroyed on new simulation run (no memory leak)

5.4 Info bubbles

- ☐ Modal opens on click of info icon (i) button
- ☐ Modal title matches selected strategy
- ☐ Modal body shows all sections: how it works, best for, scenarios, key risk/advantage
- ☐ Steady/bear/bull scenarios visually distinguished with colored left borders
- ☐ G-K key risk section displays with amber warning styling
- ☐ Close button (x) closes modal and returns focus to info icon
- ☐ Close button in footer closes modal
- ☐ Clicking overlay background closes modal
- ☐ Escape key closes modal
- ☐ Modal content updates if strategy is changed while modal is open
- ☐ Modal is accessible: role=dialog, aria-modal, aria-labelledby, focus management

